BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA: ĐIỆN - ĐIỆN TỬ

MÔN HỌC: PHÁT TRIỂN ỨNG DỤNG DI ĐỘNG



BÁO CÁO ĐỀ TÀI

HCM-UTE SMART ACCESS

HỆ THỐNG QUẢN LÝ RA VÀO PHÒNG HỌC SỬ DỤNG VÂN TAY VÀ THẺ NFC

GVHD:	ThS. Huỳnh Hoàng Hà
Mã môn học:	MBAD436364
SVTH:	**MSSV:**
Hồ Nhật Định	23119137
Hoàng Mạnh Hùng	23119150
Nguyễn Minh Hùng	23119151
Trần Xuân Minh	23119171
Đặng Minh Sơn	23119200

TP. HỒ CHÍ MINH, THÁNG 10 NĂM 2026

LỜI CẢM ƠN

Trong quá trình thực hiện đề tài “HCM-UTE Smart Access”, nhóm đã nhận được nhiều sự hướng dẫn, góp ý và hỗ trợ quý báu từ giảng viên phụ trách, ThS. Huỳnh Hoàng Hà. Những định hướng của thầy trong quá trình học tập và thực hiện đề tài, đặc biệt ở các nội dung liên quan đến phát triển ứng dụng Android, thiết kế hệ thống nhúng, cơ sở dữ liệu và bảo mật, đã tạo nền tảng quan trọng giúp nhóm từng bước xây dựng và hoàn thiện hệ thống quản lý ra vào phòng học sử dụng thẻ NFC và vân tay.

Nhóm xin chân thành cảm ơn thầy đã hỗ trợ trong quá trình phân tích yêu cầu, thiết kế kiến trúc hệ thống, xây dựng ứng dụng di động, tích hợp Firebase và triển khai kiểm thử trên ESP32-WROOM-32. Bên cạnh đó, nhóm cũng xin gửi lời cảm ơn đến các bạn đã hỗ trợ trong quá trình kiểm thử hệ thống với nhiều tài khoản khác nhau, thử nghiệm chức năng xác thực bằng NFC và vân tay, điều khiển cửa bằng servo cũng như kiểm tra chức năng điểm danh.

Mặc dù nhóm đã nỗ lực hoàn thiện hệ thống ở mức có thể triển khai và trình diễn toàn bộ quy trình từ ứng dụng di động đến thiết bị phần cứng, đề tài vẫn còn một số hạn chế nhất định về kiến trúc phần mềm, phạm vi kiểm thử, độ ổn định của kết nối mạng và khả năng mở rộng khi triển khai cho nhiều phòng học. Nhóm rất mong nhận được những ý kiến đóng góp từ giảng viên để tiếp tục cải tiến và hoàn thiện hệ thống trong các phiên bản tiếp theo.

Nhóm thực hiện đề tài

LỜI CAM ĐOAN

Nhóm thực hiện đề tài xin cam đoan rằng toàn bộ nội dung trong báo cáo "HCM-UTE Smart Access" phản ánh trung thực quá trình phân tích, thiết kế, xây dựng và kiểm thử hệ thống. Các thành phần kỹ thuật bao gồm mã nguồn ứng dụng Android, Firestore Security Rules và firmware ESP32 được sử dụng làm cơ sở để trình bày kiến trúc hệ thống, luồng dữ liệu, cơ chế xử lý và các kết quả triển khai thực tế.

Các sơ đồ, hình minh họa và tài liệu tham khảo trong báo cáo được sử dụng nhằm hỗ trợ việc mô tả và phân tích hệ thống. Những nội dung chưa được kiểm chứng đầy đủ bằng thực nghiệm hoặc chưa hoàn tất trong phạm vi đề tài đều được trình bày rõ dưới dạng hạn chế hoặc định hướng phát triển, không được xem là những kết quả đã hoàn thành.

Nhóm xin chịu trách nhiệm về tính trung thực và chính xác của các nội dung được trình bày trong báo cáo.

Nhóm thực hiện đề tài

DANH SÁCH THÀNH VIÊN THAM GIA ĐỒ ÁN

HỌ VÀ TÊN	MSSV	MỨC ĐỘ HOÀN THÀNH
Hồ Nhật Định	23119137	100%
Hoàng Mạnh Hùng	23119150	100%
Nguyễn Minh Hùng	23119151	100%
Trần Xuân Minh	23119171	100%
Đặng Minh Sơn	23119200	100%

MỤC LỤC

CHƯƠNG 1: PHẦN MỞ ĐẦU 9
1.1. Lý do chọn đề tài 9
1.2. Đối tượng nghiên cứu 9
1.3. Mục tiêu đề tài 10
1.4. Phạm vi nghiên cứu 11
1.5. Yêu cầu thiết kế 11
1.6. Phương pháp thực hiện 12
1.7. Bố cục báo cáo 13
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ NỀN TẢNG HỆ THỐNG 14
2.1. Android, Kotlin và Jetpack Compose 14
2.2. Mô hình Single-Activity và quản lý trạng thái 14
2.3. Firebase Authentication 15
2.4. Cloud Firestore và đồng bộ realtime 15
2.5. Firestore Security Rules và mô hình phân quyền 15
2.6. ESP32 WROOM-32 và các ngoại vi 16
2.7. NFC RC522 16
2.8. Cảm biến vân tay 16
2.9. Servo MG90S và LCD I2C 16
2.10. Vai trò của các nền tảng và thành phần trong hệ thống 18
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 19
3.1. Đối tượng và yêu cầu chức năng 19
3.2. Kiến trúc tổng thể 19
3.3. Phân tích kiến trúc ứng dụng Android 20
3.3.1. Mô hình kiến trúc hiện tại 20
3.3.2. Cấu trúc mã nguồn Android 21
3.3.3. Điều phối ứng dụng bằng MainActivity 23
3.3.4. Điều hướng dựa trên currentScreen 24
3.3.5. Xác thực, role và status 25
3.3.6. Quản lý state và vòng đời listener 26

3.3.7. StudentHomeScreen – trung tâm chức năng Student 28
3.3.8. Nhóm chức năng Admin 33
3.3.9. AttendanceManagementScreen và đồng bộ phiên 34
3.3.10. Quản lý thiết lập cục bộ 35
3.4. Mô hình dữ liệu Firestore 36
3.5. Firestore Security Rules 38
3.6. Luồng enrollment NFC và fingerprint 40
3.7. Luồng xác thực và mở cửa thời gian thực 41
3.8. Thiết kế phần cứng ESP32_A102 43
3.9. Đánh giá kiến trúc Android hiện tại 44
3.10. Định hướng MVVM/Repository khi mở rộng 45
CHƯƠNG 4: TRIỂN KHAI, KIỂM THỬ VÀ ĐÁNH GIÁ 47
4.1. Môi trường triển khai 47
4.2. Kết quả kiểm thử tích hợp hệ thống 47
4.3. Đánh giá kết quả đạt được 50
4.4. Hạn chế hiện tại 51
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 52
5.1. Kết luận 52
5.2. Kết quả đạt được 52
5.3. Hướng phát triển 53
TÀI LIỆU THAM KHẢO 54

DANH MỤC HÌNH ẢNH

Hình 1.1. Quy trình thực hiện đề tài HCM-UTE Smart Access 12

Hình 3.1. Kiến trúc logic tổng thể ứng dụng Android HCM-UTE Smart Access 20

Hình 3.2. Cấu trúc các thành phần chính trong app/src/main 21

Hình 3.3. Sơ đồ điều hướng dựa trên currentScreen và role/status 24

Hình 3.4. Luồng xác thực và điều hướng sau đăng nhập 25

Hình 3.5. Student Dashboard hoàn chỉnh 33

Hình 3.6. Admin Dashboard và Credential Management 34

Hình 3.7. Các collection Firestore chính của hệ thống 36

Hình 3.8. Luồng sự kiện truy cập thời gian thực 41

Hình 3.9. Sơ đồ đấu nối phần cứng 44

Hình 3.10. So sánh kiến trúc hiện tại và hướng MVVM/Repository đề xuất 45

Hình 4.1. Kết quả kiểm thử đăng ký NFC qua Android – Firebase – ESP32 49

Hình 4.2. Kết quả kiểm thử xác thực ra vào và đồng bộ trạng thái truy cập 50

DANH MỤC BẢNG

Bảng 1.1. Mục tiêu và kết quả mong đợi của đề tài 10

Bảng 2.1. Vai trò của các thành phần sử dụng trong hệ thống 18

Bảng 3.1. Đối tượng và quyền truy cập trong hệ thống 19

Bảng 3.2. Các tệp Kotlin chính và chức năng 21

Bảng 3.3. Quy tắc ánh xạ role/status sang giao diện 25

Bảng 3.4. Vai trò các collection Firestore 36

Bảng 3.5. Nhóm Firestore Security Rules chính 39

Bảng 3.6. Kết nối phần cứng ESP32_A102 43

Bảng 3.7. Đánh giá kiến trúc Android hiện tại 44

Bảng 4.1. Môi trường triển khai 47

Bảng 4.2. Kết quả kiểm thử tích hợp hệ thống 47

DANH MỤC CÁC TỪ VIẾT TẮT

Từ viết tắt	Ý nghĩa
API	Application Programming Interface - Giao diện lập trình ứng dụng
BaaS	Backend as a Service - Backend dưới dạng dịch vụ
ESP32	Vi điều khiển Wi-Fi/Bluetooth của Espressif
NFC	Near Field Communication - Giao tiếp tầm gần
RFID	Radio-Frequency Identification - Nhận dạng bằng tần số vô tuyến
UID	Unique Identifier - Mã định danh duy nhất
UI	User Interface - Giao diện người dùng
MVVM	Model - View - ViewModel
BOM	Bill of Materials - Bộ quản lý phiên bản thư viện
SDK	Software Development Kit
IoT	Internet of Things - Internet Vạn Vật

CHƯƠNG 1: PHẦN MỞ ĐẦU

1.1. Lý do chọn đề tài

Trong môi trường đại học, việc quản lý ra vào phòng học, phòng thí nghiệm và các khu vực hạn chế cần đồng thời đáp ứng yêu cầu về tính thuận tiện, an toàn và khả năng truy vết. Các phương pháp truyền thống như sử dụng chìa khóa cơ hoặc ghi chép thủ công còn nhiều hạn chế trong việc xác định chính xác người truy cập, thời điểm truy cập cũng như phương thức xác thực được sử dụng. Khi số lượng sinh viên và nhu cầu sử dụng phòng học ngày càng tăng, việc quản lý tài khoản, quyền truy cập, lịch sử ra vào và thông tin điểm danh theo thời gian thực trở thành một yêu cầu cần thiết.

Sự kết hợp giữa ứng dụng Android, nền tảng Firebase và thiết bị ESP32 tạo điều kiện xây dựng một hệ thống quản lý truy cập phù hợp với phạm vi đồ án nhưng vẫn đảm bảo khả năng tích hợp nhiều chức năng. Trong hệ thống, ứng dụng di động cung cấp giao diện riêng cho Student và Admin; Firebase Authentication đảm nhiệm xác thực tài khoản và duy trì phiên đăng nhập; Cloud Firestore được sử dụng để lưu trữ, đồng bộ dữ liệu theo thời gian thực và làm trung gian trao đổi thông tin; trong khi ESP32_A102 trực tiếp xử lý thẻ NFC, cảm biến vân tay, điều khiển servo cửa và ghi nhận các sự kiện truy cập.

Xuất phát từ nhu cầu trên, đề tài "HCM-UTE Smart Access" được xây dựng nhằm hình thành một quy trình quản lý truy cập khép kín, bao gồm đăng ký tài khoản, phê duyệt người dùng, đăng ký thông tin xác thực, xác thực tại cửa, điều khiển mở cửa, lưu lịch sử truy cập, ghi nhận điểm danh và cập nhật trạng thái theo thời gian thực trên ứng dụng. Trong phạm vi báo cáo, kiến trúc ứng dụng Android được tập trung phân tích sâu vì đây là thành phần đảm nhiệm tương tác với người dùng, điều phối các chức năng nghiệp vụ và kết nối phần lớn dữ liệu giữa người dùng, Firebase và thiết bị ESP32.

1.2. Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài là kiến trúc và quá trình triển khai hệ thống quản lý ra vào phòng A102. Hệ thống bao gồm ứng dụng Android được phát triển bằng Kotlin và Jetpack Compose; nền tảng backend sử dụng Firebase Authentication, Cloud Firestore

và Firestore Security Rules; cùng bộ điều khiển tại cửa sử dụng ESP32 WROOM-32 kết hợp đầu đọc RC522, cảm biến vân tay, màn hình LCD I2C và servo MG90S.

Báo cáo tập trung phân tích cách các thành phần trong hệ thống được tổ chức và phối hợp với nhau, trong đó nhấn mạnh kiến trúc Single-Activity của ứng dụng Android, cơ chế điều hướng dựa trên trạng thái, quản lý Compose state, đồng bộ dữ liệu thời gian thực bằng SnapshotListener, cơ chế phân quyền giữa Student, Admin và ESP32, mô hình dữ liệu trên Cloud Firestore, cũng như luồng enrollment giữa ứng dụng di động và thiết bị phần cứng.

1.3. Mục tiêu đề tài

Bảng 1.1. Mục tiêu và kết quả mong đợi của đề tài

Mục tiêu	Kết quả mong đợi
Quản lý tài khoản	Student đăng ký, tài khoản pending; Admin phê duyệt và kích hoạt.
Xác thực ra vào	Cho phép sử dụng NFC hoặc fingerprint độc lập tại ESP32_A102.
Điều khiển cửa	Servo mở khi ACCESS GRANTED và tự đóng sau 3 giây.
Quản lý credential	Student/Admin có thể đăng ký hoặc xóa NFC/vân tay thông qua luồng kiểm soát.
Theo dõi realtime	Ứng dụng phản ánh trạng thái thiết bị, cửa, access log, credential và attendance.
Điểm danh	Tạo phiên điểm danh; ESP32 ghi present/late và chống ghi trùng.
Phân quyền	Firestore Rules giới hạn dữ liệu theo role, UID, studentId và định danh thiết bị.

1.4. Phạm vi nghiên cứu

Trong phạm vi đề tài, hệ thống HCM-UTE Smart Access được triển khai và kiểm thử tập trung tại phòng A102 với một bộ điều khiển sử dụng ESP32 WROOM-32. Ứng dụng Android hỗ trợ hai vai trò người dùng chính là Student và Admin, trong đó tài khoản Student được quản lý theo hai trạng thái là pending và active nhằm bảo đảm người dùng chỉ có thể sử dụng các chức năng truy cập sau khi được Admin phê duyệt.

Hệ thống hỗ trợ hai phương thức xác thực vật lý độc lập gồm thẻ NFC và vân tay. Người dùng chỉ cần xác thực thành công bằng một trong hai phương thức để được cấp quyền truy cập, không yêu cầu sử dụng đồng thời cả hai yếu tố. Đối với dữ liệu vân tay, template sinh trắc học được lưu trực tiếp trong bộ nhớ của cảm biến; Cloud Firestore chỉ lưu trạng thái đăng ký và giá trị fingerprintSlot dùng để tham chiếu đến vị trí template tương ứng, qua đó hạn chế việc lưu trữ trực tiếp dữ liệu sinh trắc học trên nền tảng đám mây.

Các sự kiện truy cập và dữ liệu điểm danh được lưu trữ trên Cloud Firestore. Ứng dụng Android sử dụng các listener để nhận và cập nhật dữ liệu gần thời gian thực, phục vụ việc hiển thị trạng thái truy cập, lịch sử ra vào, thông tin thiết bị và kết quả điểm danh cho Student và Admin.

Phiên bản hiện tại được xây dựng dưới dạng prototype hoàn chỉnh cho một phòng học và một bộ điều khiển ESP32. Hệ thống chưa được triển khai đồng thời trên nhiều phòng học, nhiều thiết bị điều khiển hoặc số lượng lớn người dùng. Việc mở rộng kiến trúc để hỗ trợ nhiều phòng và nhiều thiết bị được xác định là một trong những hướng phát triển tiếp theo của đề tài.

1.5. Yêu cầu thiết kế

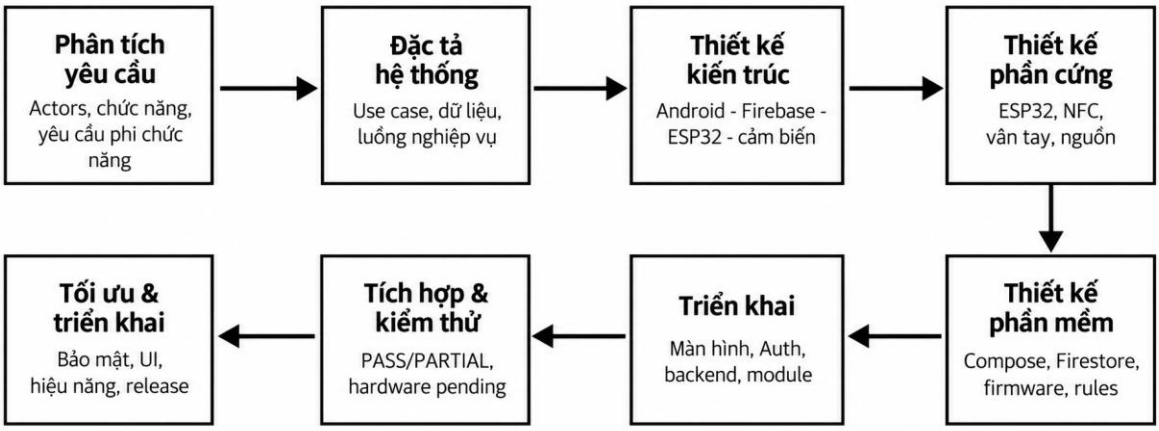
Hệ thống cần có giao diện Android trực quan, phân tách rõ chức năng Student và Admin. Student không được tự thay đổi role, status hoặc chỉnh sửa credential ngoài luồng cho phép.

ESP32 phải xác thực được NFC hoặc vân tay, hiển thị trạng thái trên LCD, điều khiển servo và ghi access log với kết quả GRANTED/DENIED, studentId, phương thức xác thực, thiết bị và timestamp.

Luồng enrollment sử dụng các trạng thái waiting, processing, completed, failed và cancelled. Chức năng điểm danh hỗ trợ cửa sổ check-in, phân loại present/late và tránh trùng bản ghi. Firestore Security Rules áp dụng deny-by-default và kiểm soát quyền theo role hoặc authUid của ESP32_A102.

1.6. Phương pháp thực hiện

Đề tài được triển khai theo hướng chia hệ thống thành các khối chức năng và kiểm thử theo từng giai đoạn. Trước tiên, nhóm xây dựng ứng dụng Android và Firebase với các chức năng đăng ký, đăng nhập, phân quyền, phê duyệt tài khoản và quản trị dữ liệu. Sau đó, hệ thống được mở rộng với các collection credentials, access_logs, devices và attendance, đồng thời tích hợp firmware ESP32 với Wi-Fi/Firebase, RC522, cảm biến vân tay, LCD và servo.



Hình 1.1. Quy trình thực hiện đề tài HCM-UTE Smart Access

Quá trình kiểm thử được thực hiện từ từng module riêng lẻ đến toàn bộ luồng end-to-end. Các chức năng NFC, enrollment vân tay, access log, điều khiển cửa và điểm danh được kiểm tra theo đúng chuỗi xử lý giữa Android, Firebase và ESP32.

1.7. Bố cục báo cáo

Báo cáo được tổ chức thành năm chương. Chương 1 trình bày tổng quan về đề tài, bao gồm bối cảnh, mục tiêu, phạm vi nghiên cứu, yêu cầu thiết kế và phương pháp thực hiện. Chương 2 giới thiệu các cơ sở lý thuyết và nền tảng công nghệ được sử dụng trong hệ thống, gồm Android, Kotlin, Jetpack Compose, Firebase và các thành phần phần cứng liên quan. Chương 3 là nội dung trọng tâm, tập trung phân tích thiết kế và kiến trúc tổng thể của hệ thống, đặc biệt là kiến trúc ứng dụng Android, cơ chế quản lý trạng thái, phân quyền và luồng dữ liệu giữa ứng dụng, Firebase và ESP32. Chương 4 trình bày quá trình hiện thực, tích hợp, kiểm thử và đánh giá kết quả của hệ thống. Chương 5 tổng kết các kết quả đạt được, chỉ ra những hạn chế còn tồn tại và đề xuất các hướng phát triển trong tương lai.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ NỀN TẢNG HỆ THỐNG

2.1. Android, Kotlin và Jetpack Compose

Ứng dụng HCM-UTE Smart Access được phát triển bằng Kotlin trên nền tảng Android. Kotlin được lựa chọn nhờ cú pháp ngắn gọn, hỗ trợ null-safety và khả năng tích hợp tốt với hệ sinh thái Android. Giao diện ứng dụng được xây dựng bằng Jetpack Compose thay cho XML layout truyền thống. Compose sử dụng mô hình khai báo, trong đó giao diện được tạo dựa trên trạng thái hiện tại; khi state thay đổi, framework tự động thực hiện recomposition để cập nhật các thành phần giao diện liên quan.

Trong mã nguồn hiện tại, các chức năng được tổ chức thành các Composable Screen riêng biệt như LoginScreen, StudentHomeScreen, AdminDashboardScreen và AttendanceManagementScreen. Material 3 được sử dụng để cung cấp các thành phần giao diện dùng chung như Card, Surface, Button, AlertDialog, Icon, typography và hệ thống màu sắc. Cách tổ chức này giúp giao diện rõ ràng, dễ phát triển và phù hợp với quy mô hiện tại của ứng dụng.

2.2. Mô hình Single-Activity và quản lý trạng thái

Ứng dụng HCM-UTE Smart Access được tổ chức theo mô hình Single-Activity, trong đó MainActivity là Activity duy nhất và các màn hình chức năng được xây dựng dưới dạng Composable. MainActivity đảm nhiệm quản lý trạng thái điều hướng currentScreen, theme, ngôn ngữ và các thiết lập cục bộ. Thay vì sử dụng Navigation Compose, ứng dụng chuyển đổi giữa các màn hình thông qua when(currentScreen) kết hợp với các callback được truyền từ từng Screen.

Trạng thái tạm thời trong các Composable được quản lý bằng remember, trong khi rememberSaveable được sử dụng cho những state cần duy trì khi Activity được tái tạo. Dữ liệu cần lưu lâu dài được lưu trên Firebase hoặc SharedPreferences. Bên cạnh đó, DisposableEffect được sử dụng để đăng ký và hủy các SnapshotListener theo vòng đời của Composable, còn LaunchedEffect được dùng cho các tác vụ phụ thuộc vào state, chẳng hạn tự động ẩn thông báo sau một khoảng thời gian.

2.3. Firebase Authentication

Firebase Authentication được sử dụng để đăng ký, đăng nhập, quản lý UID và duy trì phiên người dùng. Sau khi xác thực, ứng dụng tiếp tục đọc users/{uid} trên Cloud Firestore để xác định role và status, từ đó quyết định quyền truy cập và màn hình phù hợp.

Khi khởi động, MainActivity thực hiện trạng thái authCheck và kiểm tra FirebaseAuth.currentUser. Nếu phiên đăng nhập còn tồn tại, ứng dụng tự động điều hướng đến Student Dashboard, Admin Dashboard hoặc Pending Approval theo role/status. Phiên đăng nhập được Firebase Authentication duy trì, trong khi remember_login chỉ là thiết lập cục bộ và không lưu mật khẩu.

2.4. Cloud Firestore và đồng bộ realtime

Cloud Firestore được sử dụng làm cơ sở dữ liệu trung tâm của hệ thống. Dữ liệu được tổ chức thành các collection riêng như users, credentials, devices, enrollment_requests, nfc_lookup, fingerprint_lookup, access_logs và các collection phục vụ điểm danh. Cách tổ chức này giúp tách biệt từng nhóm nghiệp vụ và cho phép các màn hình truy cập dữ liệu phù hợp với chức năng của mình.

Để cập nhật giao diện theo thời gian gần thực, ứng dụng sử dụng SnapshotListener. Khi dữ liệu trên Firestore thay đổi, listener nhận giá trị mới, cập nhật Compose state và kích hoạt recomposition để làm mới giao diện. Cơ chế này được áp dụng cho các thông tin như trạng thái cửa, thiết bị, credential, enrollment request, access log và attendance.

2.5. Firestore Security Rules và mô hình phân quyền

Ứng dụng không chỉ ẩn chức năng ở UI mà còn áp dụng phân quyền tại Firestore Security Rules. Các hàm signedIn(), isAdmin(), isActiveStudent() và isA102Device() được dùng làm điều kiện nền. Student chỉ đọc dữ liệu phù hợp với UID/studentId của mình; Admin có quyền quản lý; ESP32_A102 được xác minh bằng authUid trong devices/ESP32_A102.

Rule cho enrollment kiểm tra action hợp lệ, userId phải trùng request.auth.uid, studentId phải đúng với hồ sơ và trạng thái ban đầu là waiting. ESP32 chỉ được cập nhật

credential hoặc lookup khi tồn tại request processing tương ứng. Cuối ruleset là default deny, giúp hạn chế truy cập ngoài các luồng đã định nghĩa.

2.6. ESP32 WROOM-32 và các ngoại vi

ESP32 WROOM-32 đóng vai trò access controller tại cửa A102. Thiết bị kết nối Wi-Fi, xác thực Firebase bằng tài khoản thiết bị, đọc RC522 và cảm biến vân tay, hiển thị hướng dẫn trên LCD, điều khiển servo và ghi dữ liệu lên Firestore. Firmware thực hiện polling enrollment request theo chu kỳ khoảng 1,5 giây, đồng thời vẫn xử lý NFC/fingerprint trong vòng lặp chính.

2.7. NFC RC522

RC522 được sử dụng làm đầu đọc thẻ NFC/RFID tại cửa phòng A102. Module kết nối với ESP32-WROOM-32 thông qua các chân SS, RST, SCK, MISO và MOSI theo cấu hình phần cứng của hệ thống. Khi người dùng đưa thẻ vào vùng đọc, RC522 cung cấp UID thẻ cho firmware. UID này được ESP32 sử dụng để tra cứu ánh xạ trong nfc_lookup và xác định credential tương ứng trước khi quyết định cấp hoặc từ chối quyền truy cập. Trong luồng enrollment, RC522 cũng được sử dụng để thu UID của thẻ mới và cập nhật kết quả đăng ký thông qua ESP32 và Cloud Firestore.

2.8. Cảm biến vân tay

Cảm biến vân tay giao tiếp với ESP32 thông qua UART. Template vân tay được lưu trực tiếp trong bộ nhớ của cảm biến, trong khi Firestore chỉ lưu trạng thái fingerprintEnrolled và vị trí fingerprintSlot để tham chiếu.

Firmware có cơ chế quét cơ sở dữ liệu của cảm biến để xác định slot còn trống dựa trên getTemplateCount, capacity và loadModel. Cách xử lý này giúp tránh gán cố định một slot cho nhiều tài khoản và hỗ trợ quản lý template vân tay linh hoạt hơn.

2.9. Servo MG90S và LCD I2C

Servo MG90S được kết nối với GPIO25 của ESP32, với góc đóng khoảng 10° và góc mở khoảng 90°. Khi xác thực thành công, hàm openDoor() điều khiển servo mở cửa

và cập nhật doorStatus trên Firestore; sau khoảng 3 giây, serviceDoor() tự động đưa cửa về trạng thái đóng.

Màn hình LCD I2C được sử dụng để phản hồi trực tiếp tại cửa, hiển thị các trạng thái như xác thực NFC/vân tay thành công, ACCESS DENIED, đăng ký NFC, CHECK-IN OK hoặc các thông báo lỗi.

2.10. Vai trò của các nền tảng và thành phần trong hệ thống

Bảng 2.1. Vai trò của các thành phần sử dụng trong hệ thống

Thành phần	Vai trò trong đề tài
Kotlin	Ngôn ngữ chính của ứng dụng Android.
Jetpack Compose + Material 3	Xây dựng UI khai báo, quản lý state và giao diện Student/Admin.
Firebase Authentication	Đăng ký, đăng nhập, UID và session.
Cloud Firestore	Lưu dữ liệu nghiệp vụ và đồng bộ realtime.
Firestore Security Rules	Phân quyền Student/Admin/ESP32, kiểm soát read/write.
ESP32 WROOM-32	Access controller, kết nối Wi-Fi, điều phối ngoại vi.
RC522	Đọc UID thẻ NFC/RFID.
Fingerprint sensor	Đăng ký và đối chiếu template vân tay theo slot.
Servo MG90S	Mô phỏng cơ cấu mở/đóng cửa.
LCD I2C 16x2	Hiển thị trạng thái cục bộ cho người dùng tại cửa.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

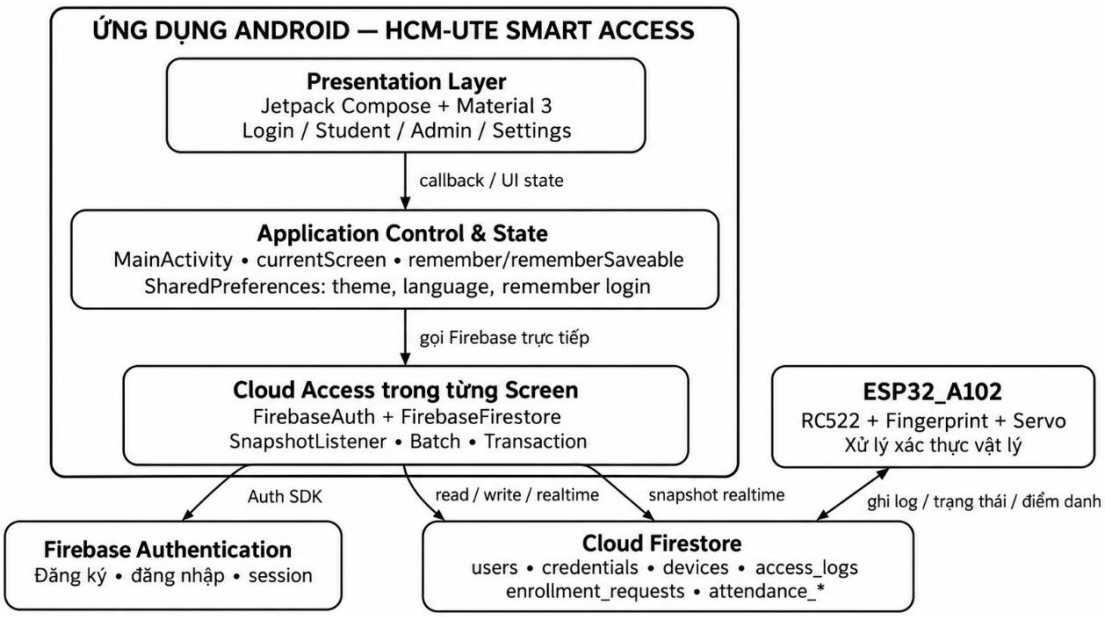
3.1. Đối tượng và yêu cầu chức năng

Bảng 3.1. Đối tượng và quyền truy cập trong hệ thống

Đối tượng	Chức năng chính	Quyền truy cập
Student	Đăng ký/đăng nhập, theo dõi profile, credential, trạng thái thiết bị, lịch sử, enrollment và attendance.	Chỉ thao tác dữ liệu của bản thân; không tự đổi role/status.
Admin	Duyệt Student, xem dashboard/log/device/credential, tạo và điều hành attendance session.	Đọc/quản lý dữ liệu toàn hệ thống theo rules.
ESP32_A102	Đọc NFC/vân tay, điều khiển servo, xử lý enrollment, tạo access log và attendance record.	Tài khoản thiết bị được xác minh bằng devices/ESP32_A102.authUid.

3.2. Kiến trúc tổng thể

Kiến trúc hệ thống được tổ chức thành ba thành phần chính gồm Android Client, Firebase Backend và Embedded Access Controller. Ứng dụng Android không giao tiếp trực tiếp với đầu đọc RC522 hoặc cảm biến vân tay. Thay vào đó, Cloud Firestore đóng vai trò trung gian để trao đổi và đồng bộ yêu cầu, trạng thái và kết quả xử lý giữa ứng dụng Android và bộ điều khiển ESP32-WROOM-32.



Hình 3.1. Kiến trúc logic tổng thể ứng dụng Android HCM-UTE Smart Access

Quyết định mở cửa không được thực hiện trực tiếp trên ứng dụng Android mà được xử lý tại firmware của ESP32. Sau khi nhận dữ liệu từ NFC hoặc cảm biến vân tay, ESP32 thực hiện tra cứu credential, xác minh trạng thái tài khoản Student và quyết định cấp hoặc từ chối quyền truy cập. Cách tổ chức này giúp chức năng điều khiển cửa hoạt động độc lập với trạng thái của ứng dụng di động. Android chủ yếu đảm nhiệm các chức năng cấu hình, quản lý, giám sát và hiển thị dữ liệu theo thời gian thực.

3.3. Phân tích kiến trúc ứng dụng Android

Ứng dụng HCM-UTE Smart Access được tổ chức theo mô hình Single-Activity với các màn hình xây dựng bằng Jetpack Compose và sử dụng Firebase làm backend. Phần này phân tích cách MainActivity điều phối màn hình, cách các Screen quản lý state và tương tác với Firestore, cũng như những ưu điểm và hạn chế của kiến trúc hiện tại.

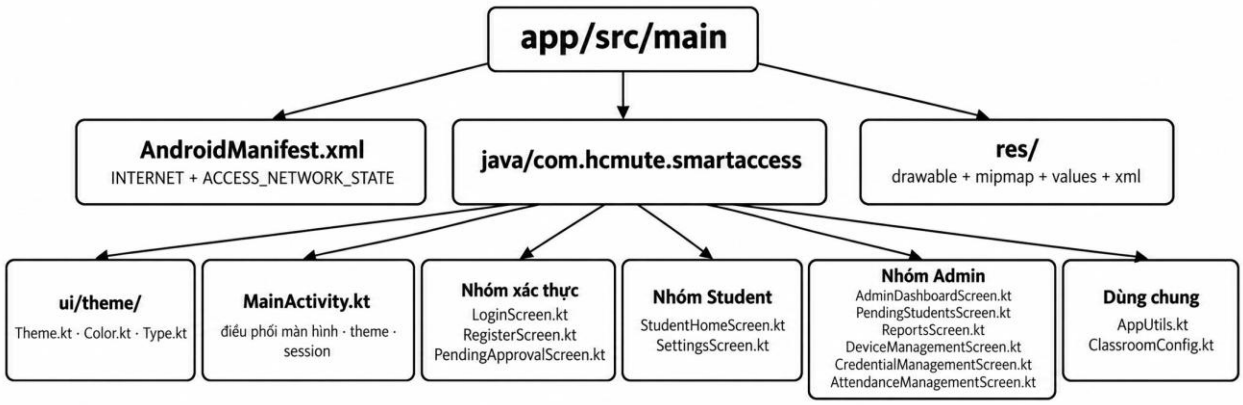
3.3.1. Mô hình kiến trúc hiện tại

Ứng dụng hiện tại chưa triển khai đầy đủ mô hình MVVM và chưa tách riêng các lớp ViewModel, Repository hoặc Data Source. Thay vào đó, hệ thống được tổ chức theo kiến trúc Single-Activity kết hợp Stateful Compose Screens và Firebase Backend-as-a-Service. MainActivity đảm nhiệm điều phối màn hình và các thiết lập chung, trong khi

mỗi Screen thực hiện đồng thời việc xây dựng giao diện, quản lý state và tương tác trực tiếp với Firebase SDK.

Cách tổ chức này giúp luồng xử lý đơn giản, dễ theo dõi và phù hợp với phạm vi prototype. Tuy nhiên, UI, business logic và lớp truy cập dữ liệu vẫn có mức độ phụ thuộc tương đối cao. Một số màn hình lớn như StudentHomeScreen phải đồng thời xử lý nhiều chức năng như SnapshotListener, enrollment, credential, access log, attendance và trạng thái cửa, làm tăng độ phức tạp khi mở rộng hệ thống hoặc thực hiện unit test.

3.3.2. Cấu trúc mã nguồn Android



Hình 3.2. Cấu trúc các thành phần chính trong app/src/main

Các tệp Kotlin chính trong cấu trúc trên được phân chia theo chức năng và vai trò xử lý cụ thể, được tổng hợp trong Bảng 3.2.

Bảng 3.2. Các tệp Kotlin chính và chức năng

Tệp nguồn	Chức năng chính
MainActivity.kt	Điểm vào ứng dụng; authCheck; điều phối currentScreen; theme, language, preference và logout.

Tệp nguồn	Chức năng chính
LoginScreen.kt	Đăng nhập Firebase Auth; đọc role/status và callback điều hướng.
RegisterScreen.kt	Tạo tài khoản Student và hồ sơ pending.
PendingApprovalScreen.kt	Theo dõi trạng thái account đang chờ Admin duyệt.
StudentHomeScreen.kt	Dashboard Student; credentials; enrollment; device/door; access log; attendance realtime.
AdminDashboardScreen.kt	Dashboard Admin; thống kê user/log/device/attendance và điều hướng màn quản trị.
PendingStudentsScreen.kt	Phê duyệt Student và khởi tạo credentials.
AccessLogsScreen.kt	Hiển thị và quản lý lịch sử ra vào.
DeviceManagementScreen.kt	Theo dõi ESP32, NFC reader, fingerprint sensor và trạng thái thiết bị.
CredentialManagementScreen.kt	Theo dõi, thu hồi NFC và gửi request xóa fingerprint.

Tệp nguồn	Chức năng chính
AttendanceManagementScreen.kt	CRUD session, open/close runtime, xem attendance records.
SettingsScreen.kt	Theme, ngôn ngữ, refresh và logout.
ClassroomConfig.kt	Hằng số ROOM_CODE/DEVICE_ID dùng thống nhất.

3.3.3. Điều phối ứng dụng bằng MainActivity

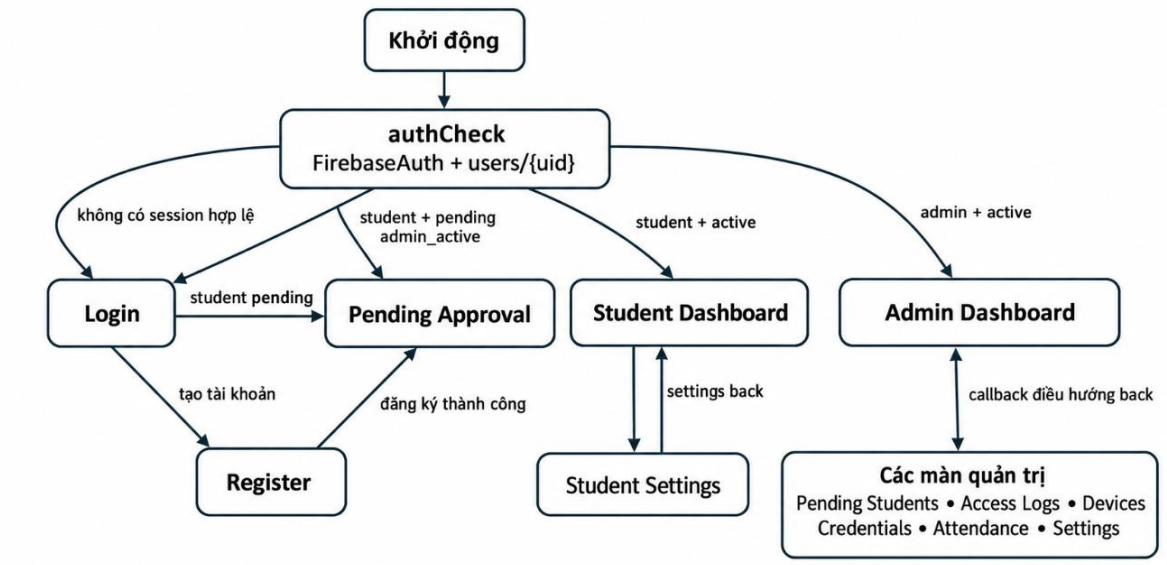
MainActivity là Activity duy nhất của ứng dụng và đóng vai trò điều phối các màn hình chính. Trong phiên bản mã nguồn hiện tại, trạng thái currentScreen được quản lý bằng rememberSaveable. Khi ứng dụng được khởi động mới, route ban đầu là authCheck; trong các trường hợp khôi phục khác, ứng dụng có thể quay về màn hình login.

Sau bước kiểm tra xác thực, ứng dụng sử dụng FirebaseAuth.currentUser kết hợp dữ liệu role và status trong users/{uid} để xác định màn hình phù hợp. Các route chính gồm login, register, pending, student, admin và các màn hình chức năng của Student hoặc Admin. Việc chuyển màn hình được thực hiện bằng cách thay đổi giá trị currentScreen thông qua callback được truyền cho từng Composable Screen.

Ngoài điều hướng, MainActivity còn quản lý các thiết lập chung như theme, ngôn ngữ và trạng thái đăng nhập cục bộ thông qua SharedPreferences. rememberSaveableStateHolder được sử dụng với currentScreen làm key để duy trì các state có thể lưu khi người dùng chuyển giữa các màn hình.

BackHandler được sử dụng để kiểm soát hành vi của nút Back theo từng route. Ví dụ, từ studentSettings người dùng được đưa về student, trong khi các màn hình quản trị như pendingStudents, accessLogs, devices, credentials và attendance quay về admin.

3.3.4. Điều hướng dựa trên currentScreen

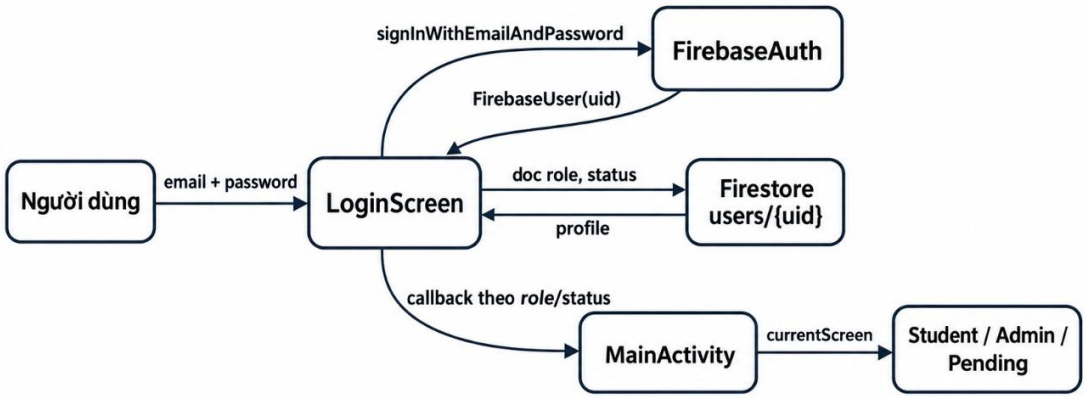


Hình 3.3. Sơ đồ điều hướng dựa trên currentScreen và role/status

Cơ chế điều hướng dựa trên state-string có ưu điểm là đơn giản, dễ triển khai và không cần bổ sung Navigation Compose. Với số lượng màn hình hiện tại, cách tổ chức này vẫn giúp luồng điều hướng tương đối dễ theo dõi.

Tuy nhiên, việc sử dụng route dạng chuỗi có thể phát sinh lỗi do sai tên route, hạn chế khi cần truyền dữ liệu phức tạp giữa các màn hình và làm số lượng callback trong MainActivity tăng dần. Khi hệ thống được mở rộng thêm nhiều phòng, thiết bị hoặc module chức năng, việc chuyển sang sử dụng sealed class hoặc Navigation Compose sẽ phù hợp hơn để tăng tính an toàn, khả năng mở rộng và bảo trì mã nguồn.

3.3.5. Xác thực, role và status



Hình 3.4. Luồng xác thực và điều hướng sau đăng nhập

Từ luồng xác thực trên, giá trị role và status của người dùng được sử dụng để xác định màn hình đích và phạm vi chức năng tương ứng, như trình bày trong Bảng 3.3.

Bảng 3.3. Quy tắc ánh xạ role/status sang giao diện

Role	Status	Màn hình đích	Ý nghĩa
admin	active	Admin Dashboard	Có quyền quản trị dữ liệu A102.
student	active	Student Dashboard	Được sử dụng chức năng Student và enrollment.
student	pending	Pending Approval	Chờ Admin phê duyệt.
Khác	không hợp lệ	Login/sign-out	Không cho truy cập dashboard.

Việc kiểm tra role và status tại giao diện giúp ứng dụng điều hướng người dùng đến đúng màn hình và giới hạn chức năng phù hợp. Tuy nhiên, cơ chế này không được

xem là lớp bảo mật duy nhất. Firestore Security Rules vẫn là thành phần chịu trách nhiệm kiểm soát quyền read/write thực tế đối với dữ liệu trên backend.

Sự kết hợp giữa điều hướng theo quyền ở phía ứng dụng và các quy tắc phân quyền ở Firestore tạo thành hai lớp kiểm soát bổ sung, giúp hạn chế truy cập sai chức năng và tăng mức độ bảo vệ dữ liệu của hệ thống.

3.3.6. Quản lý state và vòng đời listener

StudentHomeScreen thể hiện rõ cách Jetpack Compose kết hợp với cơ chế cập nhật thời gian thực của Cloud Firestore. Screen đăng ký nhiều SnapshotListener để theo dõi các dữ liệu như hồ sơ người dùng, credential, thiết bị, enrollment request, access log và attendance. Khi dữ liệu trên Firestore thay đổi, callback của listener cập nhật các biến state tương ứng, từ đó Compose thực hiện recomposition để làm mới giao diện.

Các listener được đặt trong DisposableEffect để gắn vòng đời của listener với Composable. Khi Composable rời khỏi composition, onDispose sẽ hủy listener nhằm hạn chế việc tạo listener trùng hoặc tiếp tục nhận dữ liệu khi màn hình không còn được sử dụng.

Cơ chế theo dõi thay đổi của credential trong StudentHomeScreen được triển khai bằng DisposableEffect kết hợp SnapshotListener, như thể hiện trong đoạn mã sau:

```
DisposableEffect(
    currentUser?.uid
) {
    if (
        currentUser == null
    ) {
        onDispose { }
    } else {
        val listener =
```

```
firestore
    .collection("credentials")
    .document(
        currentUser.uid
    )
    .addSnapshotListener {
            document,
            _ ->
        if (
            document != null &&
            document.exists()
        ) {
            nfcUid =
                document.getString(
                    "nfcUid"
                ) ?: ""
            nfcStatus =
                document.getString(
                    "nfcStatus"
                ) ?: "not linked"
            fingerprintEnrolled =
                document.getBoolean(
                    "fingerprintEnrolled"
                ) ?: false
            fingerprintSlot =
```

```
                    document.getLong(
                        "fingerprintSlot"
                    ) ?: 0L
                } else {
                    nfcUid = ""
                    nfcStatus = "not linked"
                    fingerprintEnrolled = false
                    fingerprintSlot = 0L
                }
            }
        onDispose {
            listener.remove()
        }
    }
}
```

Đoạn mã trên cho thấy dữ liệu trong credentials/{uid} được ánh xạ trực tiếp vào các state như nfcUid, nfcStatus, fingerprintEnrolled và fingerprintSlot. Khi document thay đổi, giao diện liên quan được cập nhật mà không cần người dùng tải lại màn hình. Listener đồng thời được hủy bằng listener.remove() khi Composable kết thúc vòng đời.

Bên cạnh DisposableEffect, ứng dụng còn sử dụng LaunchedEffect cho các tác vụ phụ thuộc state hoặc thời gian, chẳng hạn cập nhật thời gian chờ enrollment và tự động thay đổi trạng thái giao diện sau khi một thao tác hoàn tất.

3.3.7. StudentHomeScreen – trung tâm chức năng Student

StudentHomeScreen là một trong những màn hình có phạm vi xử lý nghiệp vụ lớn nhất của ứng dụng. Màn hình này hiển thị thông tin hồ sơ sinh viên, trạng thái tài khoản, credential NFC và vân tay, trạng thái thiết bị/cửa, lịch sử truy cập và thông tin điểm danh.

Đồng thời, Screen cũng đảm nhiệm việc tạo yêu cầu enrollment để ESP32 xử lý các thao tác liên quan đến credential.

Thay vì cho phép Student trực tiếp thay đổi dữ liệu credential nhạy cảm, ứng dụng tạo request trong enrollment_requests/ESP32_A102. Request chứa thông tin người dùng, thiết bị, action và trạng thái ban đầu waiting. ESP32 tiếp nhận request, chuyển sang trạng thái processing, thực hiện thao tác tương ứng trên NFC hoặc cảm biến vân tay, sau đó cập nhật kết quả completed hoặc failed. Android theo dõi document này để phản ánh trạng thái xử lý trên giao diện.

Khi Student gửi yêu cầu đăng ký hoặc thay đổi credential, ứng dụng tạo một document trong enrollment_requests với trạng thái ban đầu là waiting, như thể hiện trong đoạn mã sau:

```
val requestData =
    hashMapOf<String, Any?>(
        "deviceId" to
            ClassroomConfig.DEVICE_ID,
        "room" to
            "A102",
        "action" to
            action,
        "userId" to
            user.uid,
        "studentId" to
            studentId,
        "fullName" to
            fullName,
        "status" to
```

```
            "waiting",
        "fingerprintSlot" to
            0L,
        "credentialValue" to
            "",
        "message" to
            "",
        "requestedAt" to
            FieldValue.serverTimestamp(),
        "completedAt" to
            null
    )
firestore
    .collection(
        "enrollment_requests"
    )
    .document(
        ClassroomConfig.DEVICE_ID
    )
    .set(
        requestData
    )
    .addOnSuccessListener {
        requestSubmitting =
            false
```

```
    enrollmentState =
        StudentEnrollmentState(
            action =
                action,
            status =
                "waiting",
            userId =
                user.uid,
            requestedAt =
                Timestamp.now()
        )
}
.addOnFailureListener {
    requestSubmitting =
        false
    errorMessage =
        appText(
            language,
            "A102 device is processing another request.",
            "Thiết bị A102 đang xử lý một yêu cầu khác."
        )
}
```

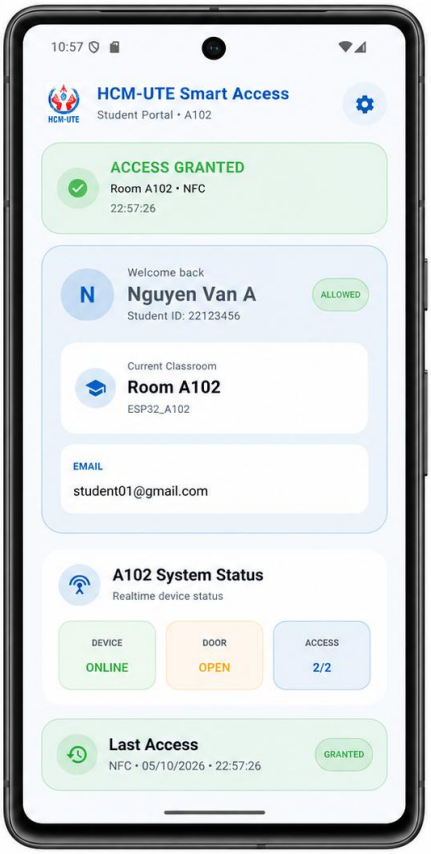
Đây là đoạn mã thực tế dùng để tạo request trên document enrollment_requests/{DEVICE_ID}. Request bao gồm deviceId, room, action, thông tin

sinh viên, trạng thái waiting, thời điểm yêu cầu và các trường phục vụ quá trình xử lý của ESP32.

Ngoài việc tạo request, StudentHomeScreen còn đăng ký SnapshotListener trên enrollment_requests/ESP32_A102 để theo dõi thay đổi trạng thái của yêu cầu. Vì vậy, khi ESP32 cập nhật request từ waiting sang processing, completed hoặc failed, giao diện Student có thể phản ánh kết quả gần thời gian thực mà không cần thực hiện polling từ phía Android.

Cách tổ chức này giúp tách thao tác vật lý trên credential khỏi ứng dụng Android. Android chịu trách nhiệm tạo và theo dõi yêu cầu, trong khi ESP32 trực tiếp xử lý cảm biến, lookup và cập nhật kết quả, phù hợp với kiến trúc Android – Firebase – ESP32 của hệ thống.

Các chức năng chính của StudentHomeScreen được thể hiện trực quan trên Student Dashboard, bao gồm thông tin cá nhân, trạng thái hệ thống A102, phương thức xác thực và lịch sử truy cập.

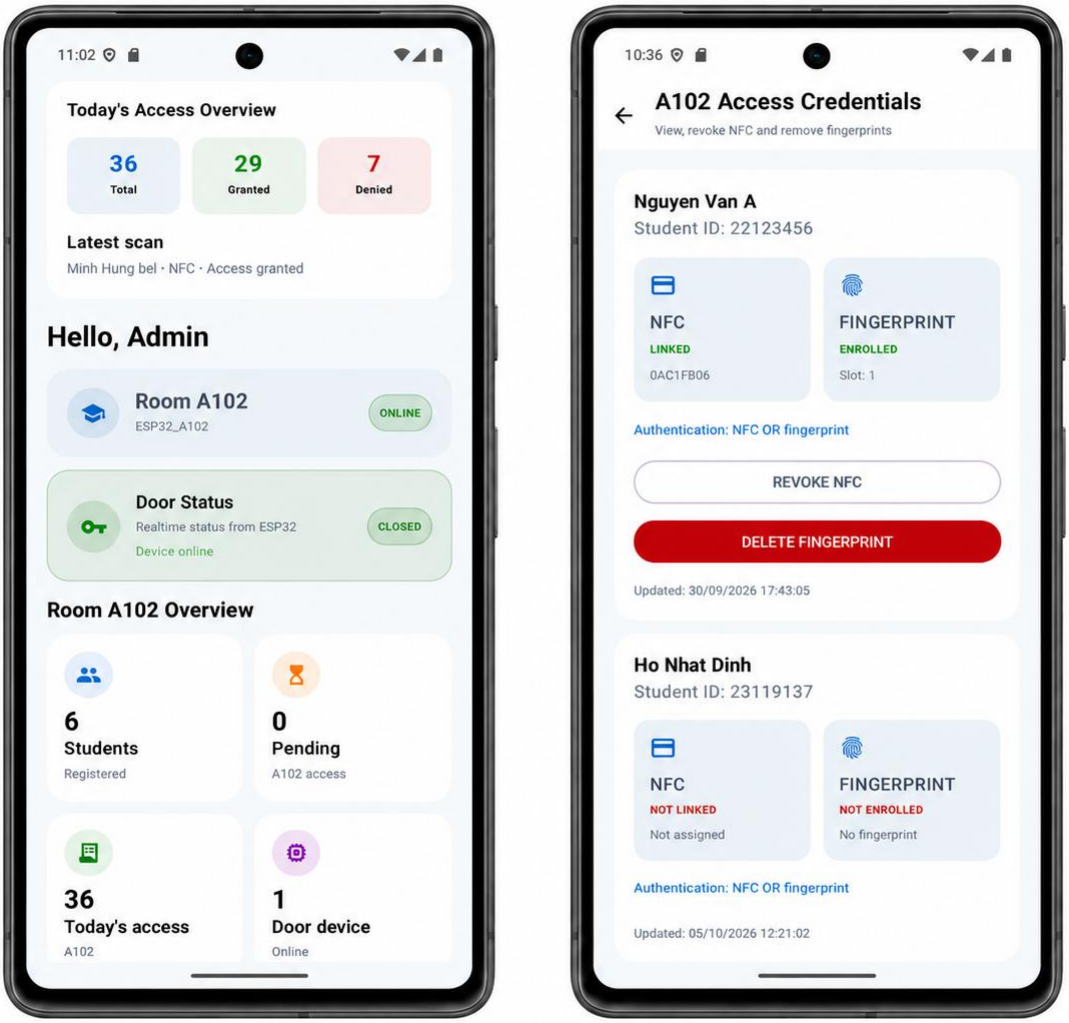


Hình 3.5. Student Dashboard hoàn chỉnh

3.3.8. Nhóm chức năng Admin

AdminDashboardScreen tổng hợp số lượng Student, pending approvals, access events, trạng thái thiết bị và phiên điểm danh. Từ Dashboard, Admin chuyển tới PendingStudents, AccessLogs, DeviceManagement, CredentialManagement, AttendanceManagement và Settings. Các màn hình này truy cập collection tương ứng và sử dụng callback quay lại Dashboard.

PendingStudentsScreen chịu trách nhiệm chuyển status của Student từ pending sang active và khởi tạo credentials. AttendanceManagementScreen có logic nghiệp vụ lớn: tạo/sửa session, mở/đóng runtime, hiển thị record và xóa session liên quan. Các thao tác nhiều document được gom bằng Firestore batch để giảm nguy cơ trạng thái không đồng bộ.



Hình 3.6. Admin Dashboard và Credential Management

3.3.9. AttendanceManagementScreen và đồng bộ phiên

AttendanceManagementScreen quản lý ba nhóm dữ liệu chính gồm attendance_sessions, attendance_runtime/ESP32_A102 và attendance_records. Trong đó, attendance_sessions lưu cấu hình của từng phiên điểm danh; attendance_runtime/ESP32_A102 lưu thông tin phiên đang hoạt động để ESP32 truy xuất; còn attendance_records lưu kết quả check-in của từng sinh viên. Khi một session được mở, ứng dụng sử dụng Firestore batch để đồng thời cập nhật trạng thái session và

runtime. Khi session được đóng hoặc xóa, runtime cũng được cập nhật tương ứng nhằm tránh việc ESP32 tiếp tục ghi nhận vào phiên cũ.

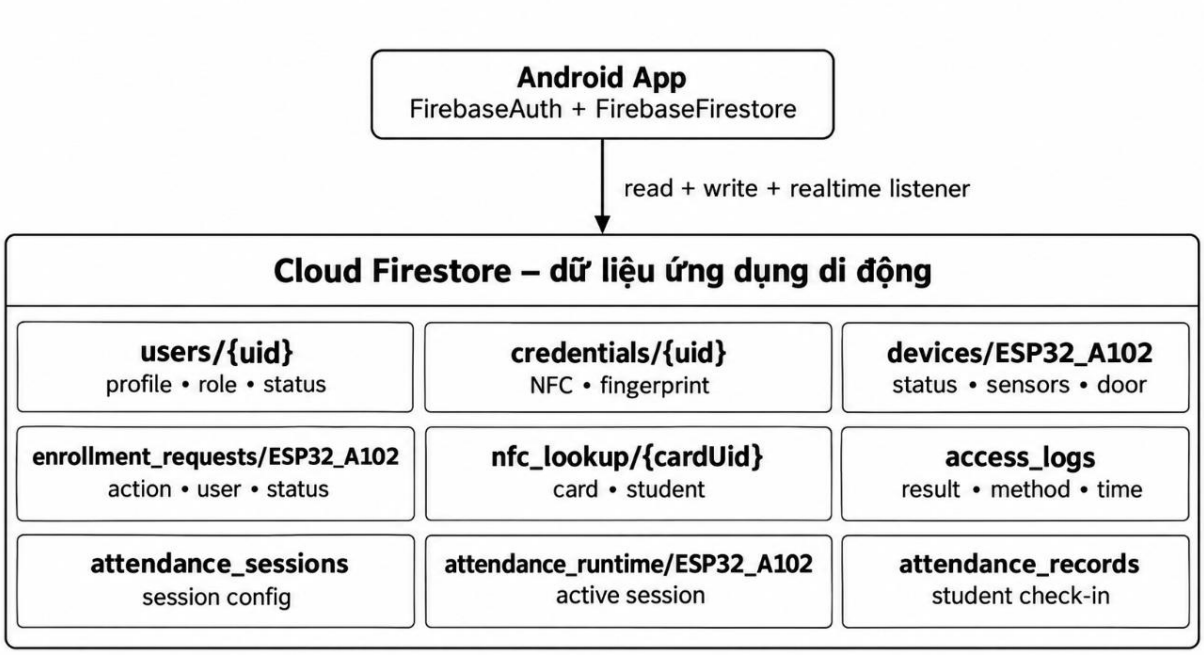
Sau mỗi lần xác thực ACCESS GRANTED, firmware kiểm tra attendance_runtime để xác định phiên điểm danh hiện hành. Nếu thời điểm hiện tại chưa đến checkInOpenAt hoặc đã vượt quá endTime, hệ thống không tạo bản ghi điểm danh. Trước khi ghi dữ liệu, ESP32 kiểm tra document có định danh theo cấu trúc sessionId_studentId để tránh tạo bản ghi trùng cho cùng một sinh viên trong một session. Trạng thái điểm danh mặc định là present; nếu thời điểm check-in vượt quá checkInCloseAt nhưng vẫn nằm trong thời gian của session, trạng thái được chuyển thành late.

3.3.10. Quản lý thiết lập cục bộ

Các thiết lập giao diện như chế độ sáng, tối hoặc theo hệ thống, cùng với ngôn ngữ sử dụng, được lưu cục bộ bằng SharedPreferences. AppLanguage và các hàm ánh xạ nội dung song ngữ được sử dụng để hiển thị văn bản tiếng Anh hoặc tiếng Việt trong các Screen, phù hợp với phạm vi prototype hiện tại mà chưa cần tổ chức toàn bộ nội dung bằng resource string riêng.

Phiên đăng nhập được Firebase Authentication duy trì. Khi ứng dụng được mở lại, MainActivity thực hiện bước authCheck để kiểm tra FirebaseAuth.currentUser, sau đó đọc thông tin role/status và tự động điều hướng người dùng đến màn hình phù hợp nếu session vẫn còn hợp lệ.

3.4. Mô hình dữ liệu Firestore



Hình 3.7. Các collection Firestore chính của hệ thống

Cấu trúc Firestore trong Hình 3.7 được triển khai thành các collection/document chuyên biệt, mỗi thành phần lưu trữ một nhóm dữ liệu và phục vụ các chức năng khác nhau, như trình bày trong Bảng 3.4.

Bảng 3.4. Vai trò các collection Firestore

Collection/Document	Dữ liệu chính	Thành phần sử dụng
users/{uid}	profile, studentId, role, status, timestamps	Login, Register, PendingStudents, Student, Attendance

Collection/Document	Dữ liệu chính	Thành phần sử dụng
credentials/{uid}	nfcUid, nfcStatus, fingerprintEnrolled, fingerprintSlot	Student, Admin credential, ESP32
devices/ESP32_A102	authUid, status, sensor status, doorStatus, lastSeen	Student, Admin, ESP32
enrollment_requests/ESP32_A102	action, userId, studentId, status, slot, credentialValue, timestamps	Student/Admin tạo request; ESP32 xử lý
nfc_lookup/{cardUid}	mapping card → user/student	ESP32 lookup/enrollment/delete
fingerprint_lookup/{slotId}	mapping slot → user/student	ESP32 fingerprint auth/enrollment/delete
access_logs	student, method, result, deviceId, timestamp	Student, Admin, ESP32 tạo

Collection/Document	Dữ liệu chính	Thành phần sử dụng
attendance_sessions	cấu hình phiên học	Admin
attendance_runtime/ESP32_A102	session active cho thiết bị	Admin, Student read, ESP32 read
attendance_records	check-in theo session/student	ESP32 tạo, Student/Admin đọc

3.5. Firestore Security Rules

Firestore Security Rules được tổ chức thành các hàm kiểm tra quyền dùng chung và các quy tắc riêng cho từng collection. Trong đó, isAdmin xác định tài khoản có role = admin và status = active, còn isActiveStudent xác định tài khoản có role = student và status = active. Đối với thiết bị ESP32_A102, hàm isA102Device xác thực bằng cách đối chiếu request.auth.uid với trường authUid trong document devices/ESP32_A102.

Cơ chế này giúp ESP32 hoạt động như một thiết bị đã được xác thực bằng Firebase Authentication thay vì truy cập Firestore công khai. Quyền đọc và ghi dữ liệu được giới hạn theo từng nhóm đối tượng và từng collection, như trình bày trong Bảng 3.5.

Bảng 3.5. Nhóm Firestore Security Rules chính

Nhóm rule	Mục tiêu kiểm soát
users	Owner đọc hồ sơ; Admin quản lý status/role.
credentials	Owner/Admin đọc; ESP32 chỉ sửa field credential khi request processing đúng user/action.
nfc_lookup	ESP32/Admin tra cứu; tạo/xóa có điều kiện theo enrollment.
fingerprint_lookup	Tương tự NFC nhưng mapping theo slot sensor.
enrollment_requests	Student active tạo request của chính mình; ESP32 chuyển trạng thái; Admin quản lý.
access_logs	Student đọc log theo studentId; ESP32 tạo log A102; Admin quản lý.
attendance_*	Admin cấu hình; ESP32 tạo record; Student chỉ đọc dữ liệu liên quan.
devices	Student/Admin đọc; ESP32 chỉ cập nhật các field trạng thái được phép.
default deny	Mọi đường dẫn không khai báo đều bị từ chối.

Đối với ESP32_A102, hệ thống sử dụng hàm isA102Device() để xác thực thiết bị dựa trên tài khoản Firebase Authentication và authUid được đăng ký trong Firestore, như thể hiện trong đoạn rule sau:

```
function isA102Device() {
  return signedIn()
    && exists(
```

```
      /databases/$(database)/documents/devices/ESP32_A102
    )
    && get(
      /databases/$(database)/documents/devices/ESP32_A102
    ).data.authUid == request.auth.uid;
}
```

Hàm trên chỉ trả về true khi yêu cầu đã được xác thực và UID của tài khoản gửi yêu cầu trùng với authUid của ESP32_A102. Nhờ đó, thiết bị chỉ có thể thực hiện các thao tác đọc/ghi được Firestore Security Rules cho phép, phù hợp với nguyên tắc deny-by-default của hệ thống.

3.6. Luồng enrollment NFC và fingerprint

Quy trình enrollment NFC và vân tay được thực hiện theo các bước sau:

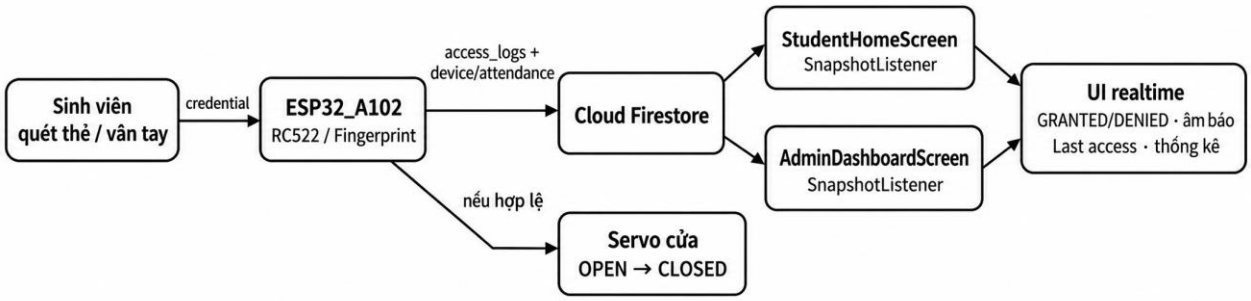
1. Student chọn ADD NFC CARD hoặc ADD FINGERPRINT trên ứng dụng. Android kiểm tra tài khoản đang ở trạng thái active và không có enrollment request khác đang được xử lý.
2. Ứng dụng tạo request tại enrollment_requests/ESP32_A102 với trạng thái waiting, kèm theo các thông tin như userId, studentId, action và thời điểm tạo yêu cầu.
3. ESP32 định kỳ kiểm tra request. Khi phát hiện request có trạng thái waiting, firmware chuyển trạng thái sang processing và bắt đầu xử lý.
4. Với NFC, ESP32 yêu cầu người dùng quét thẻ để lấy UID. Với vân tay, firmware tìm fingerprint slot còn trống và thực hiện quá trình thu nhận, tạo và lưu template trên cảm biến.
5. Sau khi xử lý thành công, ESP32 cập nhật nfc_lookup hoặc fingerprint_lookup, đồng thời cập nhật thông tin tương ứng trong credentials/{uid}.

6. Request được chuyển sang trạng thái completed hoặc failed. StudentHomeScreen theo dõi thay đổi bằng SnapshotListener và hiển thị kết quả trên giao diện.

7. Các thao tác xóa NFC hoặc vân tay cũng sử dụng cùng cơ chế request để bảo đảm dữ liệu trên cảm biến, lookup và credential được cập nhật đồng bộ.

Việc sử dụng một document điều phối cố định tại enrollment_requests/ESP32_A102 giúp firmware đơn giản hơn vì chỉ cần theo dõi một đường dẫn. Tuy nhiên, thiết kế này chỉ cho phép mỗi thiết bị xử lý một enrollment request tại một thời điểm. Khi hệ thống mở rộng nhiều thiết bị hoặc cần xử lý nhiều yêu cầu song song, có thể chuyển sang mô hình queue bằng subcollection hoặc tạo document riêng theo requestId.

3.7. Luồng xác thực và mở cửa thời gian thực



Hình 3.8. Luồng sự kiện truy cập thời gian thực

Firmware hỗ trợ hai phương thức xác thực độc lập gồm NFC hoặc vân tay. Với NFC, UID của thẻ được sử dụng để tra cứu trong nfc_lookup; với vân tay, cảm biến trả về fingerprint slot đã nhận diện và firmware tiếp tục tra cứu trong fingerprint_lookup. Sau khi xác định được danh tính người dùng, hệ thống kiểm tra trạng thái tài khoản trước khi quyết định cấp quyền truy cập.

Khi kết quả là ACCESS GRANTED, ESP32 điều khiển servo mở cửa, ghi access log và thực hiện xử lý điểm danh thông qua processAttendance(). Trường hợp ACCESS

DENIED, cửa không được mở nhưng sự kiện vẫn có thể được ghi lại để phục vụ việc theo dõi và truy vết.

Trạng thái cửa được cập nhật vào devices/ESP32_A102 thông qua các hàm openDoor() và closeDoor(). Nhờ đó, ứng dụng Student và Admin có thể theo dõi trạng thái cửa gần thời gian thực. Cách tổ chức này giúp trạng thái hiển thị trên Android phản ánh trực tiếp từ access controller thay vì chỉ được mô phỏng ở phía ứng dụng.

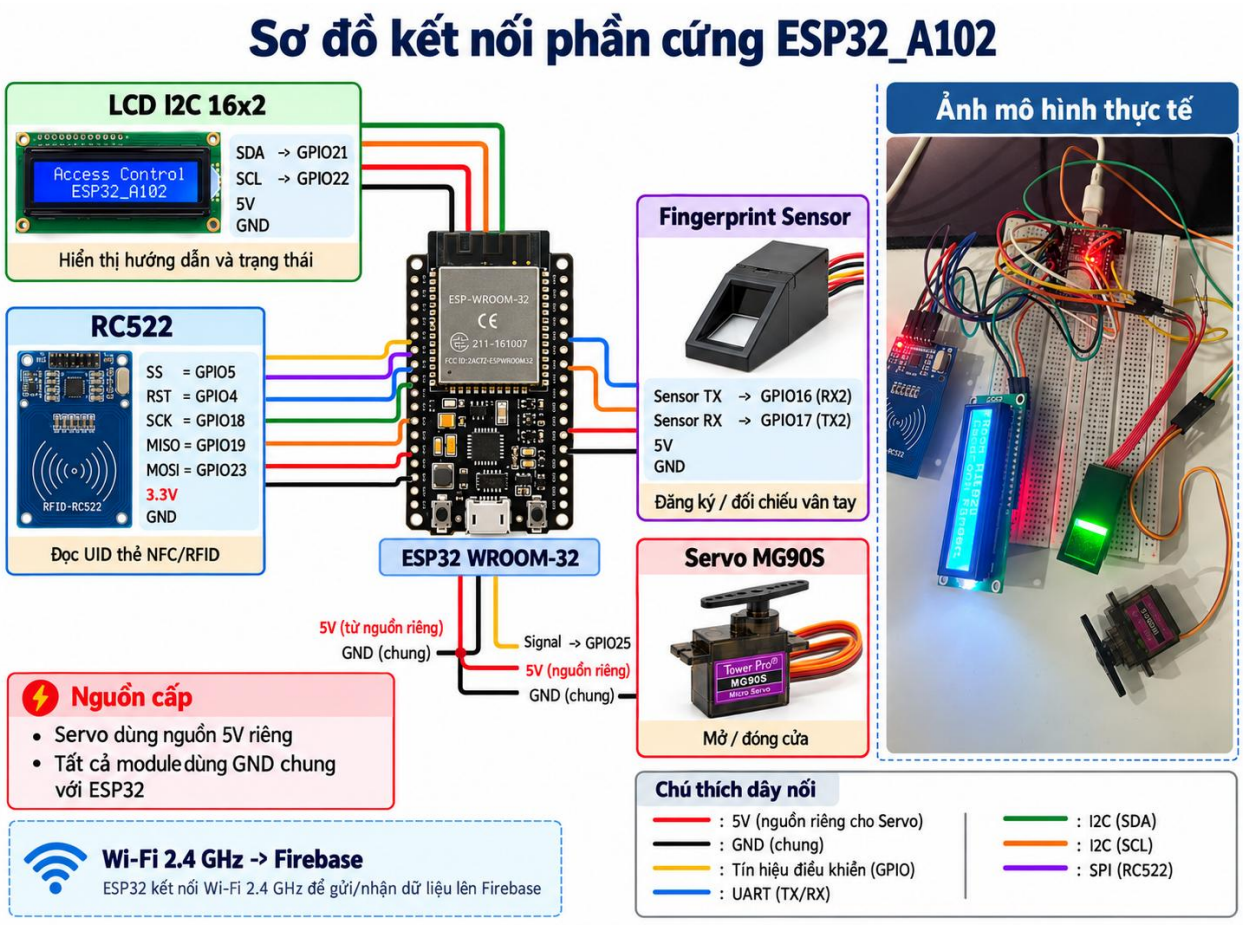
3.8. Thiết kế phần cứng ESP32_A102

Để triển khai bộ điều khiển truy cập tại phòng A102, các ngoại vi được kết nối với ESP32 theo từng giao tiếp và chân GPIO cụ thể, đồng thời đảm nhiệm các chức năng khác nhau trong hệ thống như trình bày trong Bảng 3.6.

Bảng 3.6. Kết nối phần cứng ESP32_A102

Khối	**Kết nối ESP32**	**Vai trò**
LCD I2C 16x2	SDA GPIO21, SCL GPIO22	Hiển thị hướng dẫn và trạng thái.
RC522	SS5, RST4, SCK18, MISO19, MOSI23	Đọc UID thẻ NFC/RFID.
Fingerprint sensor	Sensor TX → ESP32 RX GPIO16; Sensor RX → ESP32 TX GPIO17	Đăng ký/đối chiếu template vân tay.
Servo MG90S	Signal GPIO25; nguồn 5V riêng, chung GND	Mở/đóng cửa; đóng 10°, mở 90°, giữ 3 giây.
Wi-Fi	2.4 GHz	Kết nối Firebase.

Từ các kết nối phần cứng được tổng hợp trong Bảng 3.6, sơ đồ đấu nối thực tế giữa ESP32-WROOM-32 và các ngoại vi của hệ thống được thể hiện trực quan trong Hình 3.9.



Hình 3.9. Sơ đồ đấu nối phần cứng

3.9. Đánh giá kiến trúc Android hiện tại

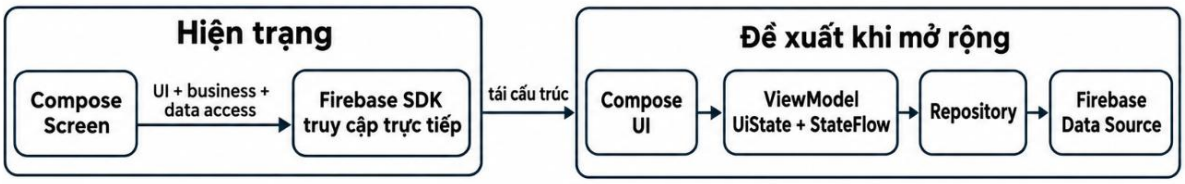
Dựa trên cách tổ chức mã nguồn và cơ chế xử lý hiện tại, kiến trúc Android của hệ thống có thể được đánh giá thông qua các đặc điểm, ưu điểm và hạn chế chính như trình bày trong Bảng 3.7.

Bảng 3.7. Đánh giá kiến trúc Android hiện tại

Đặc điểm	Ưu điểm	Hạn chế
Single-Activity + Compose	Ít boilerplate; UI tập trung; phù hợp prototype.	MainActivity quản lý nhiều route string/callback.

Đặc điểm	Ưu điểm	Hạn chế
Firebase trực tiếp trong Screen	Phát triển nhanh; luồng dữ liệu rõ khi debug.	UI-business-data liên kết chặt; khó unit test.
SnapshotListener + DisposableEffect	Realtime tốt; gắn lifecycle Compose.	Nhiều listener làm Screen lớn và khó theo dõi.
Batch/Transaction	Giảm sai lệch nhiều document.	Logic dữ liệu nằm rải rác ở Screen.
SharedPreferences	Nhẹ và đủ cho theme/language.	Có thể chuyển DataStore khi mở rộng.
Route theo role/status	Đơn giản, dễ hiểu.	Cần backend rules để bảo mật thực tế.
Enrollment qua Firestore	Android và ESP32 tách rời, realtime, dễ demo.	Một document cố định giới hạn concurrency.

3.10. Định hướng MVVM/Repository khi mở rộng



Hình 3.10. So sánh kiến trúc hiện tại và hướng MVVM/Repository đề xuất

Khi hệ thống được mở rộng, kiến trúc Android có thể được tổ chức lại theo mô hình MVVM nhằm tách biệt giao diện, logic nghiệp vụ và lớp truy cập dữ liệu. Các ViewModel như StudentViewModel, AdminViewModel và AttendanceViewModel có thể quản lý UIState thông qua StateFlow, trong khi các lớp Repository như AuthRepository, UserRepository, CredentialRepository, AttendanceRepository và DeviceRepository đảm

nhiệm việc tương tác với Firebase SDK. Cơ chế điều hướng dựa trên currentScreen dạng chuỗi cũng có thể được thay bằng Navigation Compose hoặc sealed class, đồng thời Hilt có thể được sử dụng để quản lý dependency.

Cách tổ chức này giúp giảm mức độ phụ thuộc giữa UI và dữ liệu, hỗ trợ kiểm thử và bảo trì tốt hơn. Tuy nhiên, trong phạm vi prototype hiện tại, việc triển khai đầy đủ MVVM chưa phải là yêu cầu bắt buộc.

Do đó, báo cáo cần phân biệt rõ giữa **kiến trúc hiện tại** và **kiến trúc đề xuất**. Phiên bản hiện tại vẫn sử dụng Single-Activity, Stateful Compose Screens và truy cập Firebase trực tiếp từ các Screen; MVVM và Repository chỉ được xem là hướng cải tiến khi hệ thống tiếp tục mở rộng.

CHƯƠNG 4: TRIỂN KHAI, KIỂM THỬ VÀ ĐÁNH GIÁ

4.1. Môi trường triển khai

Bảng 4.1. Môi trường triển khai

Thành phần	Phiên bản/Cấu hình
Ngôn ngữ Android	Kotlin 2.2.10
Android Gradle Plugin	9.3.2
Compose BOM	2026.02.01
Firebase BOM	34.18.0
SDK	minSdk 26; target/compile SDK 37
Java compatibility	Java 11
Thiết bị test Android	Pixel 7 emulator, Android 15 / API 35
Firmware	Arduino IDE, ESP32 Dev Module, Serial 115200
Backend	Firebase Authentication + Cloud Firestore

4.2. Kết quả kiểm thử tích hợp hệ thống

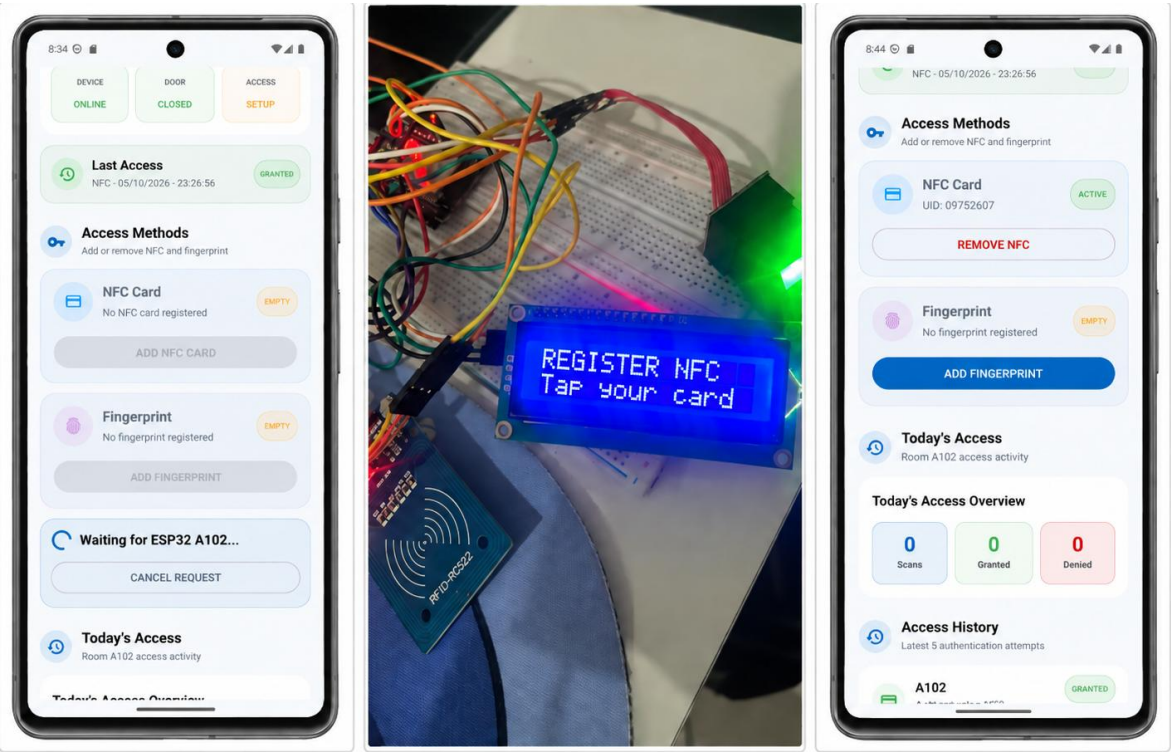
Hệ thống được kiểm thử theo luồng tổng thể từ ứng dụng Android, Firebase đến ESP32_A102. Các chức năng chính đều hoạt động đúng và dữ liệu được đồng bộ giữa ứng dụng với thiết bị.

Bảng 4.2. Kết quả kiểm thử tích hợp hệ thống

Chức năng	Kết quả	Nhận xét
Đăng ký, đăng nhập và phân quyền	Đạt	Điều hướng đúng theo role/status.

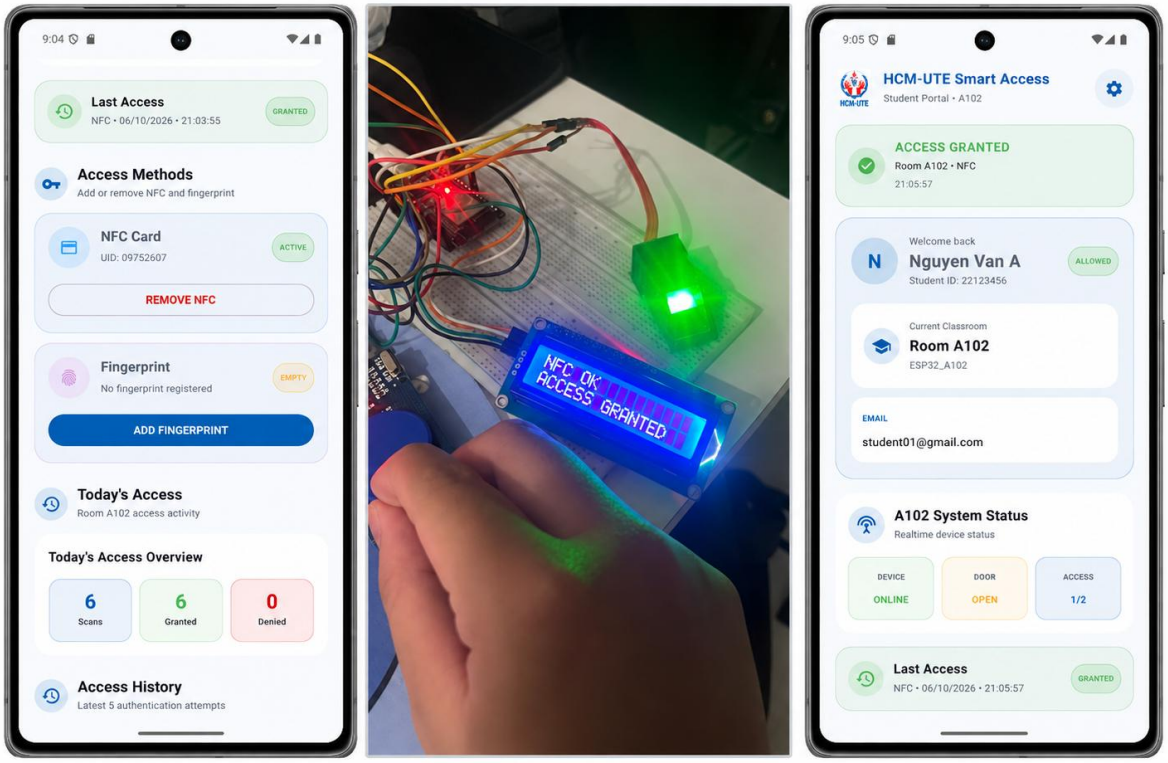
Phê duyệt tài khoản Student	Đạt	Tài khoản được kích hoạt và khởi tạo credential.
Đăng ký/xóa NFC và vân tay	Đạt	ESP32 xử lý và cập nhật Firestore đúng.
Xác thực ra vào	Đạt	NFC/vân tay được nhận diện, servo mở cửa và ghi access log.
Theo dõi trạng thái thiết bị/cửa	Đạt	Dữ liệu được cập nhật gần thời gian thực.
Điểm danh	Đạt	Ghi nhận present/late và hạn chế ghi trùng.

Để minh họa quá trình kiểm thử chức năng đăng ký credential, hệ thống được kiểm tra theo luồng từ ứng dụng Android đến Firebase và thiết bị ESP32_A102. Student gửi yêu cầu đăng ký NFC trên ứng dụng; yêu cầu được lưu trên Cloud Firestore và tiếp nhận bởi ESP32. Sau khi quá trình xử lý hoàn tất, trạng thái credential được cập nhật và đồng bộ trở lại giao diện ứng dụng. Kết quả kiểm thử được minh họa trong Hình 4.1.



Hình 4.1. Kết quả kiểm thử đăng ký NFC qua Android – Firebase – ESP32

Bên cạnh quá trình enrollment, hệ thống cũng được kiểm thử với luồng xác thực ra vào hoàn chỉnh. Khi NFC hợp lệ được nhận diện tại ESP32_A102, thiết bị thực hiện kiểm tra credential, cấp quyền truy cập, điều khiển servo mở cửa và ghi nhận access log. Nếu đang có phiên điểm danh hoạt động, hệ thống đồng thời cập nhật attendance record tương ứng. Kết quả của luồng kiểm thử này được minh họa trong Hình 4.2.



Hình 4.2. Kết quả kiểm thử xác thực ra vào và đồng bộ trạng thái truy cập

Kết quả kiểm thử cho thấy sau khi xác thực thành công, ESP32 có thể điều khiển servo mở cửa, ghi nhận access log và cập nhật trạng thái thiết bị lên Firestore. Các thay đổi được phản ánh trên ứng dụng Android thông qua cơ chế đồng bộ gần thời gian thực. Khi phiên điểm danh đang hoạt động, kết quả xác thực thành công tiếp tục được hệ thống sử dụng để ghi nhận check-in theo session tương ứng.

4.3. Đánh giá kết quả đạt được

Hệ thống đã hoàn thiện được luồng xử lý end-to-end từ ứng dụng Android đến thiết bị phần cứng. Student sau khi được Admin kích hoạt có thể đăng ký credential, ESP32 xử lý xác thực bằng NFC hoặc vân tay, điều khiển servo mở/đóng cửa, ghi access log và cập nhật attendance khi có phiên điểm danh đang hoạt động. Các thay đổi sau đó được đồng bộ về ứng dụng và phản ánh gần thời gian thực trên giao diện. Kết quả kiểm

thử với nhiều tài khoản cũng cho thấy credential được liên kết theo từng user/student, không phụ thuộc vào một tài khoản được hard-code cố định.

Về kiến trúc Android, ưu điểm nổi bật là luồng dữ liệu trực quan: các Screen đăng ký listener trực tiếp với Firestore và cập nhật Compose state khi dữ liệu thay đổi. Cách tổ chức này phù hợp với phạm vi prototype, hỗ trợ phát triển nhanh và thuận tiện trong quá trình kiểm thử, debug. Tuy nhiên, một số Screen có phạm vi xử lý lớn, logic Firebase còn phân tán và khó tách riêng cho unit test. Vì vậy, MVVM kết hợp Repository được đề xuất như hướng cải tiến cho giai đoạn mở rộng, thay vì được xem là kiến trúc đã triển khai trong phiên bản hiện tại.

4.4. Hạn chế hiện tại

+ Hệ thống hiện chỉ triển khai cho ESP32_A102 tại phòng A102, chưa hỗ trợ nhiều phòng hoặc nhiều thiết bị.
+ Enrollment sử dụng một document cố định nên chủ yếu phù hợp với xử lý tuần tự; khi có nhiều yêu cầu đồng thời cần cơ chế queue hoặc requestId.
+ Android chưa tách ViewModel/Repository, nên logic nghiệp vụ và truy cập Firebase còn nằm trực tiếp trong Composable Screen.
+ Hệ thống chưa có backend trusted riêng; Android và ESP32 truy cập Firestore trực tiếp dưới Firestore Security Rules.
+ Trạng thái online của thiết bị chưa dùng heartbeat và lastSeen để đánh giá chính xác theo thời gian.
+ Chưa thực hiện đầy đủ kiểm thử tải, mất Wi-Fi kéo dài, nhiều enrollment request đồng thời và đo độ trễ định lượng.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Đề tài HCM-UTE Smart Access đã xây dựng được hệ thống quản lý ra vào phòng học kết hợp ứng dụng Android, Firebase và ESP32. Hệ thống hỗ trợ hai vai trò Student/Admin, quản lý trạng thái tài khoản, credential NFC/vân tay, lịch sử truy cập, trạng thái thiết bị và cửa, đồng thời tích hợp chức năng điểm danh. ESP32_A102 đảm nhiệm việc xác thực tại cửa, điều khiển servo, cập nhật dữ liệu lên Firestore và phản hồi trạng thái qua LCD.

Về kiến trúc phần mềm, ứng dụng Android hiện được tổ chức theo mô hình Single-Activity kết hợp Stateful Jetpack Compose Screens và Firebase Backend-as-a-Service. Kiến trúc này phù hợp với quy mô prototype, tận dụng tốt khả năng đồng bộ gần thời gian thực của Firestore và hỗ trợ triển khai nhanh. Mô hình MVVM chưa được áp dụng trong phiên bản hiện tại mà được xác định là hướng cải tiến khi hệ thống tiếp tục mở rộng.

5.2. Kết quả đạt được

+ Hoàn thiện đăng ký, đăng nhập bằng Firebase Authentication và điều hướng theo role/status.
+ Hoàn thiện phê duyệt Student và quản lý credential.
+ Hoàn thiện đăng ký, xóa NFC/vân tay thông qua ESP32.
+ Hoàn thiện xác thực bằng NFC hoặc vân tay, điều khiển servo mở cửa 3 giây và cập nhật doorStatus.
+ Hoàn thiện access_logs và lịch sử truy cập cho Student/Admin.
+ Hoàn thiện điểm danh với session, runtime, record, trạng thái present/late và chống ghi trùng.
+ Áp dụng Firestore Security Rules phân quyền Student/Admin/ESP32 và cơ chế default deny.

+ Kiểm thử nhiều tài khoản Student sử dụng chung controller ESP32_A102.

5.3. Hướng phát triển

Để nâng cao khả năng mở rộng, bảo mật và bảo trì hệ thống trong các phiên bản tiếp theo, một số hướng phát triển được đề xuất như sau:

+ Refactor Android theo MVVM/Repository, sử dụng StateFlow và Hilt khi cần.
+ Thay điều hướng bằng chuỗi với Navigation Compose hoặc sealed route.
+ Chuyển SharedPreferences sang DataStore khi số lượng cấu hình tăng.
+ Bổ sung Cloud Functions hoặc API trusted cho thao tác đặc quyền và audit log phía server.
+ Áp dụng Firebase App Check khi phát hành chính thức.
+ Mở rộng hệ thống cho nhiều phòng, nhiều controller và chuyển enrollment sang queue theo requestId.
+ Bổ sung push notification cho các sự kiện bất thường và trạng thái thiết bị.
+ Sử dụng heartbeat và lastSeen để phát hiện thiết bị offline.
+ Mở rộng kiểm thử với thiết bị thật, mất mạng, reconnect, ESP32 restart và nhiều request đồng thời.
+ Đóng gói APK/AAB và hoàn thiện tài liệu triển khai.

TÀI LIỆU THAM KHẢO

[1] HCM-UTE Smart Access Project, “Mã nguồn ứng dụng Android HCM-UTE Smart Access,” module app, tài liệu nội bộ, phiên bản tháng 10 năm 2026.

[2] HCM-UTE Smart Access Project, “Firmware ESP32_A102,” phiên bản tích hợp NFC, cảm biến vân tay, servo và chức năng điểm danh, tài liệu nội bộ, 2026.

[3] HCM-UTE Smart Access Project, “Firestore Security Rules,” phiên bản phân quyền Student, Admin và ESP32, tài liệu nội bộ, 2026.

[4] Android Developers, “Jetpack Compose UI App Development Toolkit,” Android Developers. [Online]. Available: https://developer.android.com/compose. [Accessed: Oct. 6, 2026].

[5] Android Developers, “State and Jetpack Compose,” Android Developers. [Online]. Available: https://developer.android.com/develop/ui/compose/state. [Accessed: Oct. 6, 2026].

[6] Google, “Firebase Authentication,” Firebase Documentation. [Online]. Available: https://firebase.google.com/docs/auth. [Accessed: Oct. 6, 2026].

[7] Google, “Cloud Firestore,” Firebase Documentation. [Online]. Available: https://firebase.google.com/docs/firestore. [Accessed: Oct. 6, 2026].

[8] Google, “Get started with Cloud Firestore Security Rules,” Firebase Documentation. [Online]. Available: https://firebase.google.com/docs/firestore/security/get-started. [Accessed: Oct. 6, 2026].

[9] Espressif Systems, “ESP32-WROOM-32 Datasheet,” ver. 3.8, Sep. 16, 2026. [Online]. Available: https://www.espressif.com/en/support/documents/technical-documents/esp32-wroom-32-datasheet. [Accessed: Oct. 6, 2026].

[10] NXP Semiconductors, “MFRC522: Standard performance MIFARE and NTAG frontend,” Rev. 3.9, Product Data Sheet, Apr. 27, 2016. [Online]. Available: https://www.nxp.com/docs/en/data-sheet/MFRC522.pdf. [Accessed: Oct. 6, 2026].

[11] Adafruit Industries, “Adafruit Fingerprint Sensor Library,” GitHub Repository. [Online]. Available: https://github.com/adafruit/Adafruit-Fingerprint-Sensor-Library. [Accessed: Oct. 6, 2026].